

JavaScript

WPC

2026 年 8 月 11 日

目录

1 语言简介	2
2 语法结构	3
2.1 保留字	3
2.2 类型	3
2.3 操作符	5
2.4 语句	6
3 对象	7
3.1 普通对象	7
3.2 数组	9
3.3 函数	9
4 标准库	11
4.1 模块	11
4.2 迭代器与生成器	11
4.3 异步编程	13
4.4 元编程	13

1 语言简介

严格来说，JavaScript 是 Sun Microsystems（现已并入 Oracle）的注册商标，最初用于指代 Netscape（现 Mozilla）对该语言的实现，而这门语言的标准化工作由 Ecma International 负责。由于商标限制，其标准版本被命名为 ECMAScript（简称 ES）。

2015 年，ES6 发布，标志着一个重要的转折点。从此之后，ECMAScript 规范改为按年发布，并以年份作为版本号（ES2016、ES2017……）。为了确保向后兼容，标准规定即便某个特性存在严重问题，也不能随意移除。不过，程序可以选择开启严格模式（strict mode）。在严格模式下，一些过时或不安全的语法将被禁止。严格模式既可以通过在代码开头添加 “use strict” 显式启用，也可能因为使用某些新的语言特性而隐式触发。

随着 JavaScript 的应用规模不断扩大，它的一些固有局限性逐渐显现出来：缺乏静态类型检查、大型项目中重构困难、IDE 支持不够精确等。为了解决这些问题，Microsoft 于 2012 年推出了 TypeScript，主要设计者是 Anders Hejlsberg（Delphi 和 C# 的核心设计师）。TypeScript 是 JavaScript 的严格超集，在 JS 的基础上添加了可选的静态类型系统——支持结构子类型、联合类型、交叉类型和泛型等，通过编译器（tsc）转换为纯 JavaScript，类型检查完全发生在编译期，不会在运行时产生额外开销。

2 语法结构

2.1 保留字

as	async	await	break	case	catch	class
const	continue	debugger	default	delete	do	else
enum	export	extends	false	finally	for	from
function	get	if	import	in	implements	instanceof
interface	let	new	null	of	package	private
protected	public	return	set	static	super	switch
target	this	throw	try	typeof	var	void
while	with	yield				

2.2 类型

在 JavaScript 中, 类型分为原始类型和对象类型, 对象类型是可修改的 (mutable), 而原始类型是不可修改的 (immutable)。原始类型包括: 数值 (Number)、字符串 (String)、布尔值 (Boolean)、null、undefined、Symbol 和 BigInt。对象类型则更为丰富, 例如: 数组 (Array)、集合 (Set)、映射 (Map)、正则表达式 (RegExp)、日期 (Date)、错误对象 (Error) 等。此外, TypeScript 在此基础上提供了更丰富的类型工具, 如工具类型 (Utility Types), 详见 [TypeScript 官方文档](#)。

2.2.1 数值

在 JavaScript 中, 十六进制字面量以 0x 或 0X 开头; 整数还可以使用 二进制 或 八进制 表示, 分别以前缀 0b 和 0o (或 0B、0O) 开头。浮点字面量支持科学计数法, 数值后可以跟字母 e 或 E, 表示该数值乘以 10 的相应指数次幂。BigInt 字面量写作一串数字后跟小写字母 n, 也可以通过前缀 0b、0o 或 0x 来表示二进制、八进制或十六进制的 BigInt。虽然标准运算符可用于 BigInt, 但不能将 BigInt 与常规数值混合运算。此外, 数值字面量中可以使用下划线对数字进行分隔, 以提高可读性。

JavaScript 通过内置 Math 对象提供了一组算术函数和变量, 详细可以参考 [MDN Web Docs](#)。在 JavaScript 中, 算术运算即使发生溢出或除以零, 也不会抛出错误。当运算结果超出可表示的数值范围 (上溢出) 时, 结果会为 Infinity 或 -Infinity。下溢出发生在运算结果比可表示的最小数值更接近零的情况下, 此时返回 0 或 -0, 而 -0 与 0 在大多数情况下几乎无法区分。

0 除以 0 是无意义的运算, 其结果为 NaN (Not a Number)。此外, 无穷大除以无穷大、负数开平方, 或者对无法转换为数值的非数值操作数进行算术运算, 结果也都是 NaN。NaN 不等于任何值, 也不等于它自身。要判断一个变量是否为 NaN, 可以使用表达式 `x !== x`, 或者调用 `Number.isNaN(x)`。

2.2.2 字符串

JavaScript 使用 UTF-16 编码，字符串由无符号 16 位值的序列组成。在 UTF-16 中，码点超过 16 位的字符会被编码为两个 16 位值的序列。然而，JavaScript 的大多数字符串操作方法通常是以 16 位值 为单位进行处理，而不是以完整字符为单位。不过如果使用 for...of 循环或展开操作符 (...) 迭代字符串，则迭代的单位是完整字符（code point）。

在 JavaScript 中，Unicode 转义字符以 \u 开头，后跟 4 位十六进制数字。为了支持码点大于 16 位的字符，可以使用花括号语法：\u{XXXXXX}。字符串字面量是单引号、双引号或反引号包裹的字符串，普通字符串字面量可以在每行末尾加一个反斜杠来实现多行书写，而使用反引号定义的模板字面量 支持跨行字符串，会保留换行符，并且可以通过 \${} 插入任意表达式。

2.2.3 符号

Symbol 类型没有字面量，必须通过 Symbol() 或 Symbol.for() 函数来创建。Symbol() 接受一个可选的字符串参数作为描述，每次调用都会返回一个唯一的 Symbol 值，永远不会重复。Symbol.for() 会在全局符号注册表中查找指定的字符串键，如果存在则返回对应的 Symbol，否则会创建一个新的 Symbol 并注册到全局表中。

2.2.4 解构赋值

解构赋值允许从数组或对象中提取值，并将其赋值给变量。对于数组，可以使用方括号语法来解构，对于对象，则可以使用花括号语法来解构，例如：

```
let [a, b] = [1, 2]; // a = 1, b = 2
[x, y] = [y, x]; // 交换 x 和 y 的值
let {name, age} = {name: "Alice", age: 30}; // name = "Alice", age = 30
```

解构赋值时，左侧多余变量会被设为 undefined，右侧多余变量则会被忽略，如果想收集所有剩余元素，可以使用剩余参数语法。

```
let [a, b, ...rest] = [1, 2, 3, 4]; // a = 1, b = 2, rest = [3, 4]
let {x, y, ...others} = {x: 1, y: 2, z: 3}; // x = 1, y = 2, others = {z: 3}
```

解构赋值还支持嵌套结构和默认值，例如：

```
let [a, [b, c]] = [1, [2, 3]]; // a = 1, b = 2, c = 3
let {x, y = 10} = {x: 5}; // x = 5, y = 10
```

2.2.5 多余属性

在 TypeScript 中，当对象字面量赋值给一个类型标注或接口时，编译器会执行多余属性检查（Excess Property Check），拒绝字面量中包含目标类型未定义的属性。这一机制可以捕获常见的拼写错误和类型误用。

```
interface Point { x: number; y: number; }  
const p: Point = { x: 1, y: 2, z: 3 }; // 错误：对象字面量只能指定已知的属性
```

需要注意的是，多余属性检查仅针对对象字面量。如果一个对象先通过其他途径获得了额外属性，再赋值给类型变量，或者通过类型断言绕过检查，编译器不会报错。若确实需要传递额外属性，可以使用索引签名或联合类型来声明，也可以通过类型断言（**as**）来绕过多余属性检查：

```
interface Point { x: number; y: number; }  
const p = { x: 1, y: 2, z: 3 } as Point; // 使用 as 绕过检查
```

此外，还可以通过中间变量赋值或索引签名来规避检查：

```
// 方式一：中间变量  
const obj = { x: 1, y: 2, z: 3 };  
const p: Point = obj; // 合法，obj 不是对象字面量  
  
// 方式二：索引签名  
interface PointWithExtra { x: number; y: number; [key: string]: unknown; }  
const p2: PointWithExtra = { x: 1, y: 2, z: 3 };
```

2.3 操作符

2.3.1 条件式

在 JavaScript 中，`null` 和 `undefined` 是唯一两个没有属性的值，当对它们使用普通的属性访问表达式时，就会抛出 `TypeError`。为避免这种错误，可以使用可选链操作符（`?.` 或 `?.[]`）。当左侧表达式的结果为 `null` 或 `undefined` 时，整个访问表达式会直接返回 `undefined`。类似地，在调用函数时，可以使用 `?()` 替代普通的 `()` 调用。如果函数本身为 `null` 或 `undefined`，那么表达式也会返回 `undefined`。

2.3.2 关系操作符

JavaScript 有两个操作符用于测试两个值是否相等。一个是 `==` 操作符，这个操作符先做类型转换，再判断相等；另一个是严格相等操作符 `===`，如果两个值不是同一种类型，那么这个操作符就不会判定它们相等。

in 操作符左侧操作数是可以转换为字符串的值，右侧操作数是对象。如果左侧的值是右侧的对象的属性名，则 **in** 返回 `true`。**instanceof** 操作符左侧操作数是对象，右侧操作数是对象类的标识。这个操作符在左侧对象是右侧类的实例时求值为 `true`。

2.3.3 其他操作符

typeof 操作符的返回值是一个字符串，表明操作数的类型。**delete** 操作符用于删除对象的属性或数组的元素，它只能作用于属性访问表达式。**await** 操作符期待一个 `Promise` 对象作为其唯一

操作数，返回值是 Promise 对象的兑现值，await 只能出现在已经通过 async 关键字声明为异步的函数中。

先定义 (first-defined) 操作符 ?? 求值其先定义的操作数：如果其左操作数不是 null 或 undefined，就返回该值。否则，它会返回右操作数的值。?? 是短路的，有些时候 ?? 是对 || 的一个有用的替代，选择先定义的操作数。例如：

```
let max = maxWidth || preferences.maxWidth || 500; // 这种用法忽略了0值
let max = maxWidth ?? preferences.maxWidth ?? 500; // max可以为0
```

2.4 语句

2.4.1 for 语句

for/of 循环专门用于遍历可迭代对象。字符串可以逐个字符迭代的，对于字符串，它会逐个迭代其中的字符；对于 Set，循环体会对集合中的每一个元素执行一次；对于 Map，迭代器会依次返回键/值对（一个数组），其中第一个元素是键，第二个元素是对应的值。

for/in 循环可以用于任意对象，用于迭代对象的属性。for/in 循环并不会枚举对象的所有属性，比如它不会枚举符号属性，对于字符串的属性，它只会遍历可枚举的属性。JavaScript 核心定义的各种内部方法是不可枚举的，不过默认情况下，手写代码定义的所有属性和方法都是可枚举的。如果 for/in 循环的循环体删除一个尚未被枚举的属性，则该属性就不会再被枚举了。

2.4.2 debugger 语句

debugger 语句本身通常什么也不做。在实际运行中，它的作用类似于在代码中设置了一个断点：当 JavaScript 执行到该语句时，程序会暂停运行，我们就可以通过调试器来打印变量的值、检查调用栈等。需要注意的是，debugger 并不会主动打开调试器，它只是在调试器已启用的情况下才会生效。比如在浏览器中，如果已经打开了开发者工具控制台，执行到 debugger 时程序就会暂停。

2.4.3 函数声明

function 声明用于定义函数，函数声明会创建一个函数对象，并把这个函数对象赋值给指定的名字，然后在程序的任何地方都可以通过这个名字来引用这个函数。位于任何 JavaScript 代码块中的函数声明都会在代码运行之前被处理，无论在作用域中的什么地方声明函数，这些函数都会被“提升”，就好像它们是在该作用域顶部定义的一样。于是在程序中，调用函数的代码可能位于声明函数的代码之前。

3 对象

3.1 普通对象

对象是属性的无序集合，每个属性都由一个名字和一个值组成。此外，每个属性还具有三个 属性特性 (property attributes)：

- writable (可写)：指定属性的值是否可以被修改。
- configurable (可配置)：指定属性是否可以被删除，以及其特性是否可以被重新定义。
- enumerable (可枚举)：指定属性名是否会在 for/in 循环中被枚举出来。

3.1.1 创建对象

对象可以通过对象字面量、new 关键字和 Object.create() 函数来创建。使用 new 操作符时，会创建并初始化一个新对象，new 后必须跟随一个构造函数调用。使用 Object.create() 时，会创建一个新对象，并将其第一个参数作为新对象的原型。这种方式常用于创建继承自某个对象的实例，同时避免对原型对象本身的直接修改。

通过对象字面量创建的对象，都共享同一个原型对象，即 Object.prototype。使用 new 关键字和构造函数创建的对象，则会以该构造函数的 prototype 属性作为它们的原型。例如，通过 new Array() 创建的对象，其原型就是 Array.prototype。几乎所有对象都有原型，但只有少数对象拥有 prototype 属性。正是这些带有 prototype 属性的对象，决定并定义了其他对象所继承的原型链结构。

3.1.2 操作属性

查询不存在的属性不是错误，而是返回 undefined。delete 操作符用来删除属性，不过只能删除自有属性，不能删除继承属性。类似的，hasOwnProperty() 方法用于测试对象是否有给定名字的属性，也只能测试自有属性。

枚举属性：

- for/in 循环，遍历所有可枚举属性。
- Object.keys() 返回可枚举自有属性名的数组，不包含名字是符号的属性。
- Object.getOwnPropertyNames() 返回所有自有属性名的数组，不包含名字是符号的属性。
- Object.getOwnPropertySymbols() 返回所有自有属性的符号名的数组。
- Reflect.ownKeys() 返回所有属性名的数组。

3.1.3 扩展对象

`Object.assign()` 接收两个或多个对象作为其参数。它会修改并返回第一个参数，对于之后的参数，它会把该对象的可枚举自有属性复制到目标对象。它按照参数列表顺序逐个处理对象，第一个对象的属性会覆盖目标对象的同名属性，而第二个对象的属性会覆盖第一个对象的同名属性。

将属性从一个对象分配到另一个对象的一个原因是，如果有一个默认对象为很多属性定义了默认值，可以将这些默认属性复制到另一个对象中。

```
o = Object.assign({}, defaults, o); //将defaults的属性复制到o中
o = {...defaults, ...o}; //与上式等价
```

3.1.4 序列化对象

对象序列化 (serialization) 指的是将对象转换为字符串的过程。在 JavaScript 中，可以使用 `JSON.stringify()` 将对象序列化为字符串，再通过 `JSON.parse()` 将其还原为对象。`JSON.stringify()` 只会序列化对象的可枚举自有属性。`JSON` (JavaScript Object Notation) 是一种数据格式，其语法是 JavaScript 语法的一个子集。

可被序列化和恢复的值包括：对象、数组、字符串、有限数值、`true`、`false` 和 `null`。`NaN`、`Infinity` 和 `-Infinity` 会被序列化为 `null`。`Date` 对象会被序列化为 ISO 格式的日期字符串，但 `JSON.parse()` 只会将其恢复为普通字符串，而不会还原为 `Date` 对象。函数、`RegExp`、`Error` 对象以及 `undefined` 无法被序列化或恢复。

3.1.5 对象字面量语法

对象字面量支持简写语法、计算属性名和扩展操作符。

```
let x = 1, y = 2;
//下式等价于 {x: x, y: y, area: function() { return x * y }}
let o = {x, y, area(){ return x * y}};

const property_name="p";
let o2 = {[property_name]: 42}; //等价于 {p: 42}

let position = {x: 1, y: 2};
let dimensions = {width: 100, height: 75};
//下式等价于 {x: 1, y: 2, width: 100, height: 75}
let rect = {...position, ...dimensions};
```

除了数据属性之外，JavaScript 还支持为对象定义访问器属性 (accessor property)，访问器属性并不是一个存储值的属性，而是由两个方法组成：获取方法 (getter) 和设置方法 (setter)。当程序读取该属性的值时，会调用获取方法，其返回值就是属性访问表达式的结果。当程序为该属性赋值时，会调用设置方法，并将赋值语句右侧的值作为参数传入。


```
let o = {  
  a: 0,  
  get next() { return this.a++; },  
  set next(n) { this.a = n; }  
};
```

3.2 数组

大多数数组方法都支持一个回调函数参数，该函数会依次接收三个参数：当前元素的值、元素的索引以及整个数组。

`flatMap()` 与 `map()` 类似，但它会在映射的同时自动进行打平操作。调用 `a.flatMap(f)` 的效果等同于（但效率远高于）`a.map(f).flat()`。你可以将 `flatMap()` 理解为更灵活的 `map()`：它不仅能把输入数组中的一个元素映射为一个结果，还可以映射为多个结果，甚至是一个空数组。如果某个元素被映射为空数组，那么在打平后，该元素就不会出现在输出数组中。

在 JavaScript 中，类数组对象并不会继承 `Array.prototype`，因此无法直接调用数组的方法。不过可以借助 `Array.prototype.call()` 来调用数组方法，或者使用 `Array.from()` 方法将类数组对象转换为真正的数组。

3.3 函数

3.3.1 定义函数

函数可以通过函数声明、函数表达式和箭头函数来定义。函数声明的定义会被提升到顶部，因此如果你不需要提升定义，请使用其他两种定义方式。

```
// 函数声明  
function f(x) {  
  return x * x;  
}  
  
// 函数表达式  
const f = function(x) {  
  return x * x;  
};  
  
// 箭头函数  
const f = (x) => x * x;
```

3.3.2 函数调用

在 JavaScript 中，`this` 关键字不像普通变量那样具有作用域机制，嵌套函数不会自动继承外层函数的 `this` 值。为了在方法调用中正确捕获 `this`，可以使用箭头函数，它会从定义时的上下文中继承 `this`，或者在调用嵌套函数时显式使用 `bind()`，将所需的 `this` 绑定到函数上。

此外，`call()` 和 `apply()` 方法可以间接调用一个函数，使其在指定对象的上下文中执行，就好像该函数是该对象的方法一样。需要注意的是，箭头函数的 `this` 是在定义时绑定的，它继承自外层作用域，无法通过 `call()`、`apply()` 或 `bind()` 改写。

JavaScript 函数支持参数解构。

```
function f({x, y}) {  
    return x * x + y * y;  
}  
f({x: 3, y: 4}); // 25
```

4 标准库

4.1 模块

在浏览器中，可以通过 `<script type="module">` 来启用模块支持。JavaScript 的模块导出主要有两种形式：命名导出（named exports）和默认导出（default exports）。这两种方式在导入时对应的语法也不同，除了静态导入之外，JavaScript 还提供了动态导入，可实现按需加载或异步加载模块。

此外，模块还支持再导出（re-export），可以将多个文件的导出集中整合到一个入口文件中，类似于 C 语言中的头文件，用于简化模块的统一管理和引用。

```
// 命名导出
export function f() { ... }
export const pi = 3.14;
// 导入命名导出
import {f, pi} from 'module';

// 默认导出
export default function() { ... }
// 导入默认导出
import anyName from 'module';

// 动态导入
import('module').then(module => { ... });

// 再导出
export {f, pi} from 'module';
```

在 JavaScript 模块中，`import.meta` 这个特殊语法引用一个对象，这个对象包含当前执行模块的元数据。其中，这个对象的 `url` 属性是加载模块时使用的 URL（在 Node 中是 `file://URL`）。`import.meta.url` 的主要使用场景是引用与模块位于同一目录下的图片、数据文件或其他资源。

```
const imageUrl = new URL('./image.png', import.meta.url);
```

4.2 迭代器与生成器

4.2.1 迭代器

在 JavaScript 中，迭代器本质上是一个对象，该对象必须实现一个 `next()` 方法，每次调用都会返回一个迭代结果对象（`IteratorResult`），迭代结果对象包含两个属性：`value`：当前迭代的值；`done`：一个布尔值，表示迭代是否结束。若要让一个类可迭代，必须实现一个名为 `Symbol.iterator`

的方法，并返回一个迭代器对象。此外，迭代器还可以定义 `return()` 方法，用于在迭代提前结束时执行清理操作，该方法需要返回一个迭代结果对象。

```
class Range {
  constructor(start, end) {
    this.start = start;
    this.end = end;
  }
  [Symbol.iterator]() {
    let current = this.start;
    const end = this.end;
    return {
      next() {
        if (current <= end) {
          return { value: current++, done: false };
        } else {
          return { value: undefined, done: true };
        }
      }
    }

    // 使迭代器本身也是可迭代的
    [Symbol.iterator]() {
      return this;
    }
  };
}
```

4.2.2 生成器

在 JavaScript 中，生成器函数使用 `function*` 定义。调用生成器函数时，并不会立即执行函数体，而是返回一个生成器对象。只有在调用生成器对象的 `next()` 方法时，函数体才会从当前位置开始执行，直到遇到 `yield` 语句为止。`yield` 表达式产生的值会作为 `next()` 返回结果对象中的 `value`。同时，`yield` 表达式本身的值，则由调用 `next()` 时传入的参数决定。

生成器函数还可以使用 `yield*` 语句，将迭代过程委托给另一个可迭代对象，实现嵌套迭代。此外，生成器函数也可以显式返回一个值，该返回值会出现在迭代结束时 `next()` 方法返回结果对象的 `value` 属性中。

```
class Range {
  *[Symbol.iterator]() {
```

```
    for (let current = this.start; current <= this.end; current++) {  
        yield current;  
    }  
}  
}
```

4.3 异步编程

回调函数是最基本的异步编程方式，通过将一个函数作为参数传递给另一个函数，当异步操作完成时，调用该回调函数来处理结果。`setTimeout()`、`setInterval()` 和各种事件注册函数是常见的基于回调的异步函数。

另一种实现异步的方式是使用返回 `Promise` 对象的函数。`Promise` 构造函数接受一个函数作为参数，该函数本身接收两个函数参数：`resolve` 和 `reject`。当异步操作成功时调用 `resolve(value)` 返回结果，当异步操作失败时调用 `reject(Error)` 返回错误。也可以使用 `Promise.resolve(value)` 或 `Promise.reject(Error)` 来快速创建 `Promise` 对象。

使用 `Promise` 可以通过链式调用 `.then()` 和 `.catch()` 来处理异步结果和错误：两者都返回一个新的 `Promise` 对象，支持链式调用。如果需要并行执行多个异步操作，可以使用 `Promise.all()`，该函数接收一个包含多个 `Promise` 对象的数组并返回一个新的 `Promise`，当所有 `Promise` 都成功时，该 `Promise` 解析为一个包含所有结果的数组；如果任意一个 `Promise` 被拒绝，该 `Promise` 会立即拒绝，返回第一个失败的原因类似的函数还有 `Promise.allSettled()`、`Promise.race()` 和 `Promise.any()`。

将函数声明为 `async` 意味着该函数的返回值会被自动包装成一个 `Promise`。需要注意的是，`await` 关键字不会阻塞整个程序的执行，它只能在使用 `async` 声明的函数内部使用，用于等待 `Promise` 的结果。

由于 `Promise` 只适合单次运行的异步计算，JavaScript 提供了 `for/await` 循环，用于迭代异步迭代器。异步可迭代对象实现了 `Symbol.asyncIterator()` 方法，该方法返回一个异步迭代器对象，其 `next()` 方法返回一个 `Promise`。异步生成器函数使用 `async function*` 定义，可以在函数体内使用 `await` 和 `yield`。

4.4 元编程

4.4.1 属性描述符

在 JavaScript 中，对象的属性具有四个内部特性，数据属性包括：`value`、`writable`、`enumerable` 和 `configurable`；访问器属性则由：`get`、`set`、`enumerable` 和 `configurable` 组成。这些特性通过一个称为属性描述符 (property descriptor) 的对象来表示。可以使用 `Object.getOwnPropertyDescriptor()` 获取某个自有属性的属性描述符；使用 `Object.defineProperty()` 可以定义或修改属性的描述符。

```
const o = { a: 1 };  
const descriptor = Object.getOwnPropertyDescriptor(o, 'a');  
console.log(descriptor);
```

```
// { value: 1, writable: true, enumerable: true, configurable: true }  
Object.defineProperty(o, 'b', {  
  value: 2,  
  writable: false,  
  enumerable: true,  
  configurable: false  
});
```

此外，JavaScript 还提供了三种方法来限制对象的可变性：`Object.preventExtensions()`：禁止为对象添加新属性；`Object.seal()`：在 `preventExtensions` 的基础上，同时将现有属性标记为不可配置；`Object.freeze()`：在 `seal` 的基础上，再将所有数据属性标记为不可写。这三种方法作用逐级递进，使对象的结构和属性逐步变得更加固定与不可更改。

4.4.2 Symbol

- `Symbol.iterator` 和 `Symbol.asyncIterator` 符号可以让对象变成可迭代对象。
- `Symbol.species` 符号可以指定派生对象的构造函数。
- `Symbol.hasInstance` 符号可以自定义 `instanceof` 运算符的行为。
- `Symbol.toStringTag`、`Symbol.toPrimitive`、`Symbol.isConcatSpreadable` 符号可以控制对象在 `Object.prototype.toString()`、`valueOf()`、`Array.prototype.concat()` 中的行为。
- `Symbol.match`、`Symbol.replace`、`Symbol.search` 和 `Symbol.split` 符号可以自定义字符串方法的行为。

4.4.3 Reflect

`Reflect` 对象提供了一组静态方法，用于模拟核心语言语法的行为。

常规语法

```

o[name]
o[name] = value
delete o[name]
name in o
Object.defineProperty(o, name, desc)
Object.getOwnPropertyDescriptor(o, nm)
Object.getPrototypeOf(o)
Object.setPrototypeOf(o, proto)
Object.preventExtensions(o)
Object.isExtensible(o)
Object.keys(o) (部分行为)
func.apply(o, args)
new Target(...args)

```

Reflect API

```

Reflect.get(o, name)
Reflect.set(o, name, value)
Reflect.deleteProperty(o, name)
Reflect.has(o, name)
Reflect.defineProperty(o, name, desc)
Reflect.getOwnPropertyDescriptor(o, nm)
Reflect.getPrototypeOf(o)
Reflect.setPrototypeOf(o, proto)
Reflect.preventExtensions(o)
Reflect.isExtensible(o)
Reflect.ownKeys(o)
Reflect.apply(func, o, args)
Reflect.construct(Target, args)

```

4.4.4 Proxy

Proxy 对象用于创建一个对象的代理，创建 Proxy 需要两个参数：目标对象 (target) 和处理程序对象 (handler)，代理对象支持的操作就是反射 API 定义的那些操作。设 p 是一个代理对象，当使用 delete 操作符删除代理对象上的一个属性时，代理对象会在处理器对象上查找 deleteProperty() 方法。如果处理器对象上不存在对应方法，则代理就在目标对象上执行基础操作。

如果处理器对象是空的，那代理本质上就是目标对象的一个透明包装器，透明包装器在创建“可撤销代理”时有用，创建可撤销代理一般不使用 Proxy() 构造函数，而要使用 Proxy.revocable() 工厂函数。这个函数返回一个对象，其中包含代理对象和一个 revoke() 函数。一旦调用 revoke() 函数，代理立即失效。

```

const handler = {
  get(target, prop, receiver) {
    console.log(`Getting property ${prop}`);
    return Reflect.get(target, prop, receiver);
  },
  set(target, prop, value, receiver) {
    console.log(`Setting property ${prop} to ${value}`);
    return Reflect.set(target, prop, value, receiver);
  }
};

const target = { a: 1, b: 2 };
const proxy = new Proxy(target, handler);
console.log(proxy.a); // Getting property a \n 1

```

```
proxy.b = 3;           // Setting property b to 3
```